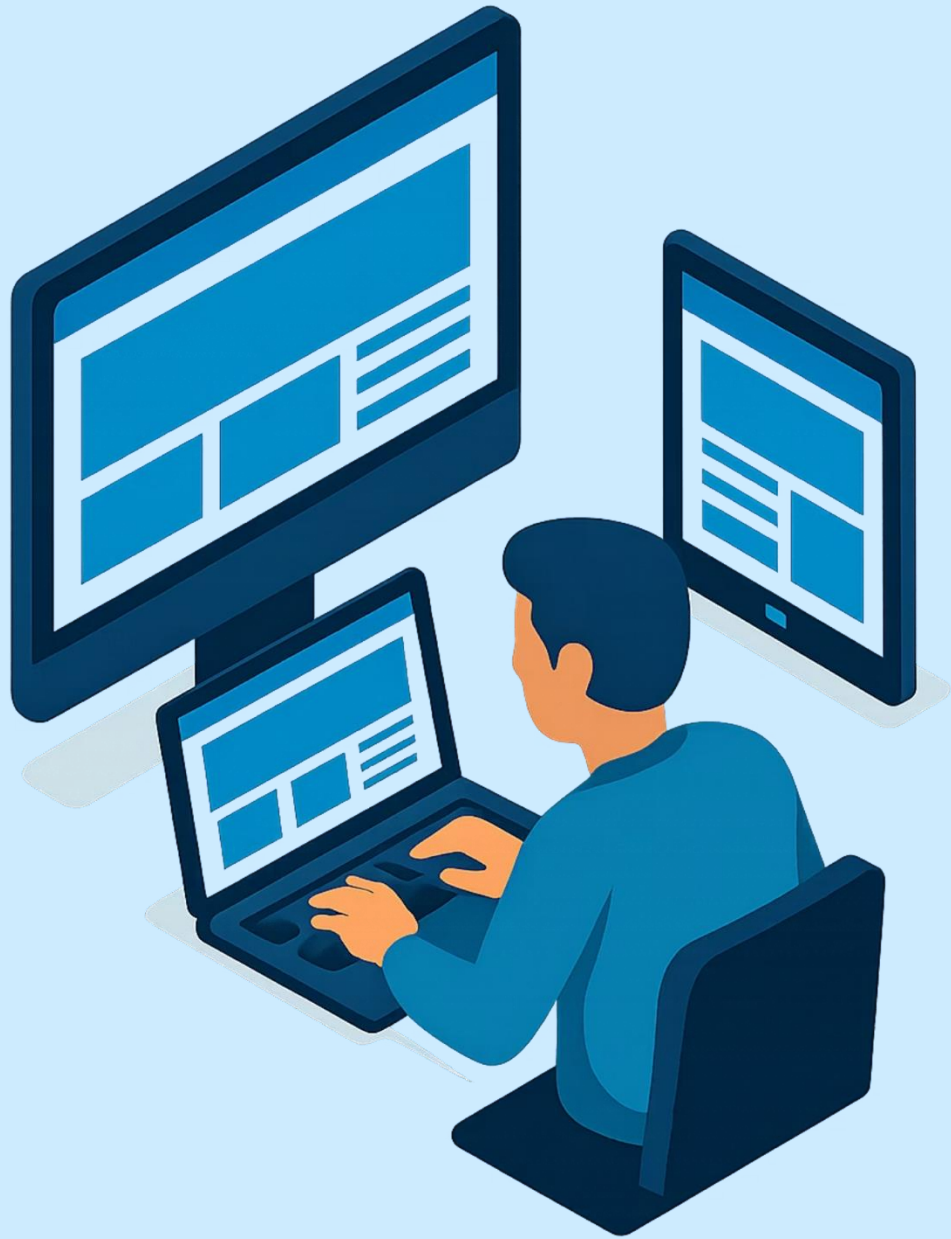


Angular Moderno

23 - Responsividade e UX

2026

SENAI - CRTI



Angular Moderno

23 - Responsividade e UX

Objetivo da aula: Evoluir a interface da aplicação para um padrão responsivo e melhorar a experiência do usuário, sem alterar nenhuma lógica existente.

Contexto do Projeto

Situação Atual

- Aplicação funcional com Angular Standalone
- Angular Material já aplicado
- Listagem de produtos com componentes Angular Material (mat-card)
- Header com navegação
- Carrinho funcional com Signals

Problema Atual

- Interface ainda não responsiva
- Layout não adaptado para mobile
- Espaçamento e organização podem melhorar

Objetivo desta Aula

- Tornar a interface responsiva
- Melhorar organização visual
- Aplicar boas práticas de UX



Conceitos Fundamentais

Responsividade

Capacidade da interface se adaptar a diferentes tamanhos de tela: mobile, tablet e desktop.

Mobile-First

Desenvolver pensando primeiro no mobile e depois expandir para telas maiores.

UX (User Experience)

- Espaçamento adequado
- Hierarquia visual
- Clareza nas ações
- Organização da informação

 **Importante:** Responsividade não é Angular! É CSS e organização de layout.

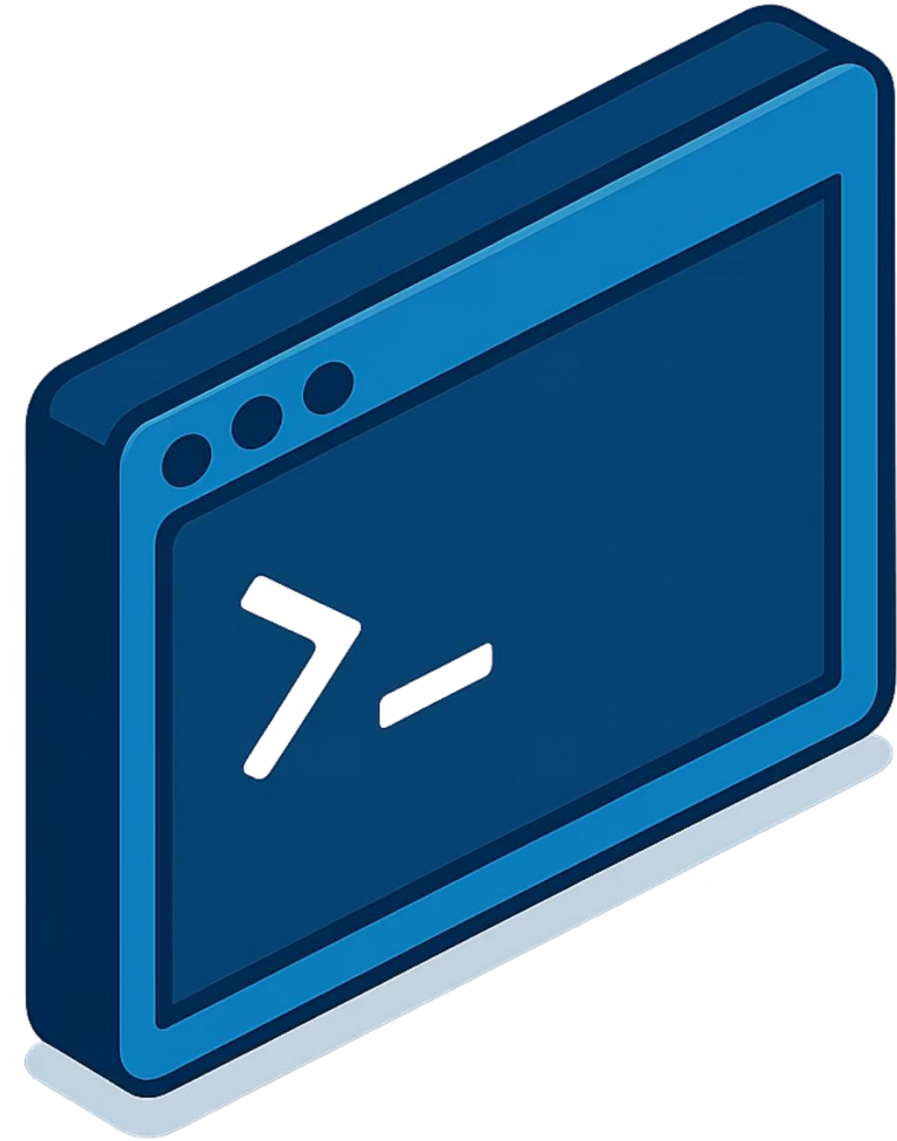
Etapa 0: Parar o Servidor

Antes de iniciar alterações estruturais, execute no terminal:

`CTRL + C`

Justificativa

- Evita inconsistências durante as alterações
- Garante que alterações de layout sejam aplicadas corretamente



Etapa 1: Organização do Layout Global

APP.HTML

Substituir a estrutura principal pelo novo layout base:

```
<app-header></app-header>

<main class="container-principal">
  <!-- CONTROLES TEMPORÁRIOS DE LOGIN -->
  <section class="bloco-controle">
    <button (click)="login()">Login</button>
    <button (click)="logout()">Logout</button>
    <p>Status: {{ usuarioLogado() ? 'Logado' : 'Deslogado' }}</p>
  </section>

  <!-- CONTEÚDO DINÂMICO DAS ROTAS -->
  <router-outlet></router-outlet>
</main>
```

app-header fixo

Passa a ser fixo na aplicação

router-outlet

Controla o conteúdo dinâmico

Estrutura base

Criação de estrutura profissional

Etapa 2: Estilização Global

APP.CSS

Adicionar os seguintes estilos:

```
.container-principal {  
  padding: 16px;  
  max-width: 1200px;  
  margin: 0 auto;  
}  
  
.bloco-controle {  
  margin-bottom: 16px;  
}
```

O que cada regra faz

- **padding: 16px** — espaçamento interno
- **max-width: 1200px** — define largura máxima
- **margin: 0 auto** — centraliza o conteúdo
- **margin-bottom** — melhora legibilidade

Etapa 3: Estrutura Responsiva da Lista

LISTA-PRODUTOS.HTML Alterar a listagem para a nova estrutura preparada para grid:

```
<!-- Estado 3: Sucesso -->
@if (!carregando() && !erro()) {
  <p>Total de produtos: {{ totalProdutos() }}</p>
  <hr />
  <section class="grid-produtos">
    @for (item of produtos(); track $index) {
      <app-produto
        [nome]="item.nome"
        [preco]="item.preco"
        (produtoSelecionado)="exibirProduto($event)"
        (produtoAdicionado)="adicionarAoCarrinho($event)"
      />
    }
  </section>
}
```

-  Substitui o layout linear por uma estrutura preparada para grid, criando a base para a responsividade que será aplicada via CSS nas próximas etapas.

Etapa 4: Grid Mobile (Base)

LISTA-PRODUTOS.CSS

Adicionar o CSS base para o grid:

```
.grid-produtos {  
  display: grid;  
  grid-template-columns: 1fr;  
  gap: 16px;  
}
```

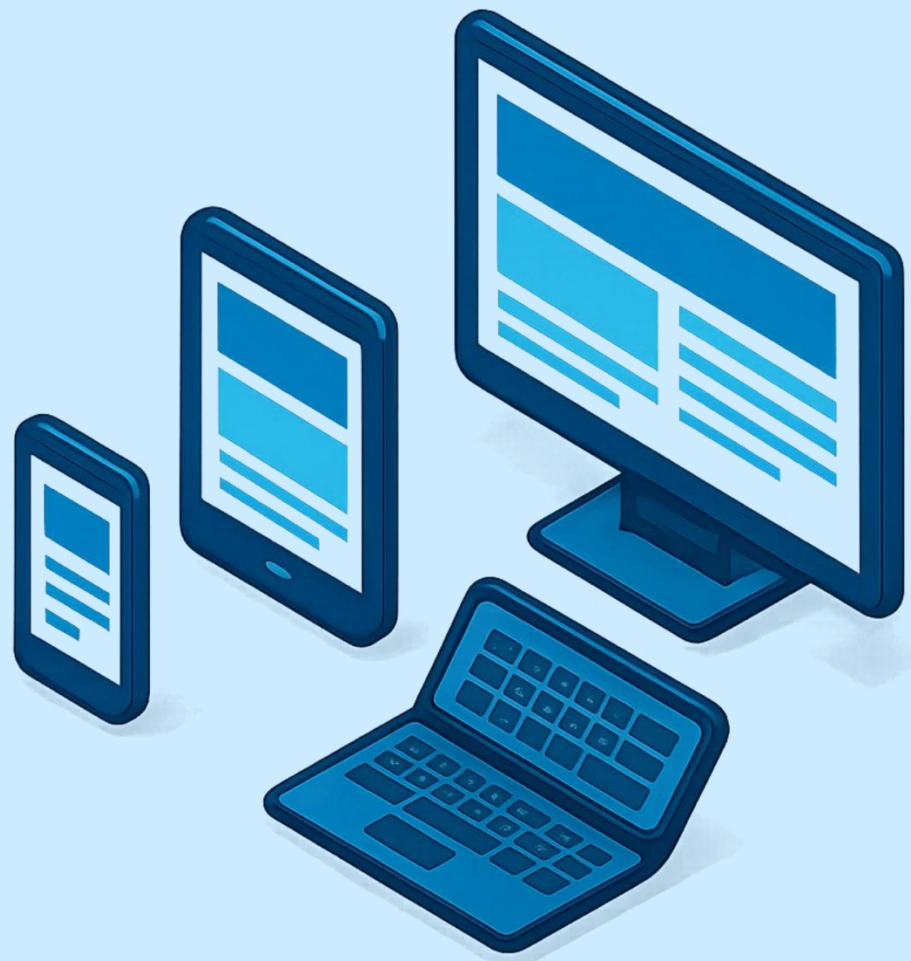
Resultado no Mobile

1 Coluna

Layout em coluna única,
otimizado para mobile

gap: 16px

Espaçamento adequado entre
os itens do grid



Etapa 5: Responsividade com Breakpoints

LISTA-PRODUTOS.CSS

Adicione os seguintes estilos ao mesmo arquivo:

```
@media (min-width: 600px) {  
  .grid-produtos {  
    grid-template-columns: repeat(2, 1fr);  
  }  
}  
  
@media (min-width: 900px) {  
  .grid-produtos {  
    grid-template-columns: repeat(3, 1fr);  
  }  
}
```

1

Mobile

Abaixo de 600px — 1 coluna

2

Tablet

600px → 2 colunas

3

Desktop

900px → 3 colunas

Etapa 6: Ajuste Visual dos Cards

PRODUTO.CSS

Substituir todo o CSS do componente por:

```
.produto {  
  margin: 8px;  
  height: 100%;  
}  
  
mat-card {  
  display: flex;  
  flex-direction: column;  
  justify-content: space-between;  
  height: 100%;  
}
```

Benefícios

- Padroniza a altura dos cards
- Mantém alinhamento visual entre colunas
- Melhora a organização geral do layout
- Cards com alturas iguais ficam mais profissionais

Etapa 7: Ajustes no Header

HEADER.CSS

Substituir todo o conteúdo do arquivo por este:

```
mat-toolbar {  
  display: flex;  
  flex-wrap: wrap;  
}  
  
button {  
  margin-left: 8px;  
}  
  
.flex-spacer {  
  flex: 1 1 auto;  
}
```

Por que isso importa?

flex-wrap: wrap

Evita quebra de layout no mobile

margin-left

Botões ficam melhor distribuídos

Adaptação

Header se adapta ao tamanho da tela

Etapa 8: Ajustes Globais

STYLES.CSS

Adicionar ao arquivo de estilos globais:

```
body {  
  margin: 0;  
  font-family: Roboto, sans-serif;  
}  
  
h2 {  
  margin-top: 16px;  
}
```

Impacto de cada regra

- **margin: 0** — remove margens padrão do navegador
- **font-family: Roboto** — padroniza tipografia com Material Design
- **margin-top: 16px** — melhora espaçamento dos títulos

Etapa 9: Rodar o Projeto

Com todas as alterações salvas, execute no terminal:

```
ng serve
```

Validações Esperadas

- Interface carrega corretamente sem erros
- Nenhum erro exibido no console do navegador
- Layout responsivo funcionando conforme esperado



Testes de Responsividade

Abrir o **DevTools** com **F12** e ativar o modo de dispositivo responsivo.



Mobile

Verificar layout em coluna única, botões acessíveis e header sem quebra.



Tablet

Confirmar grid de 2 colunas a partir de 600px e espaçamento adequado.



Desktop

Validar grid de 3 colunas a partir de 900px e largura máxima de 1200px.



Conceito Importante: Separação de Responsabilidades

O que NÃO foi alterado

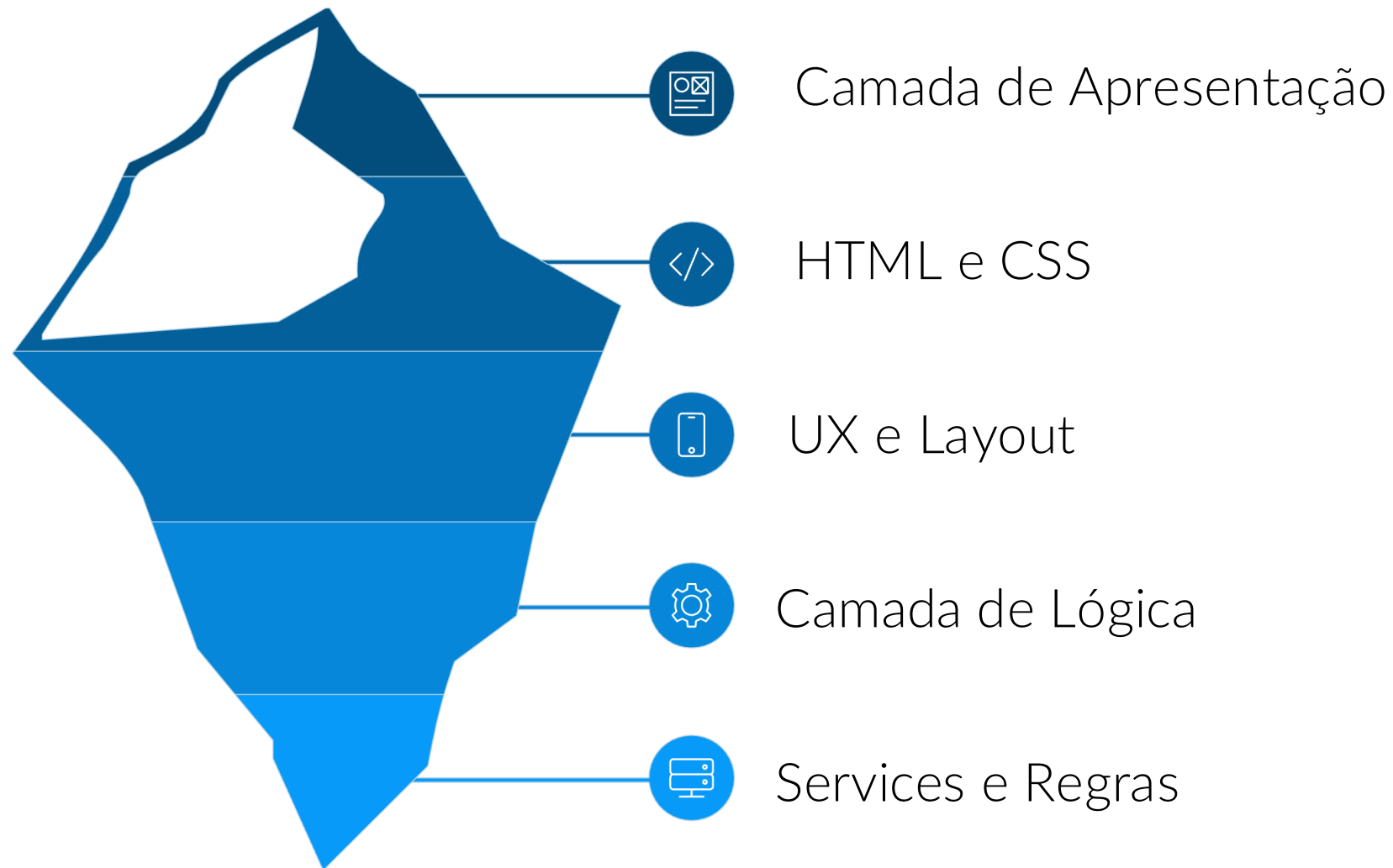
- Lógica de negócio
- Services
- Signals
- Comunicação entre componentes

O que foi alterado

- Apenas a interface (HTML + CSS)

Nenhuma alteração foi feita em arquivos TypeScript nesta aula.

Esta separação é uma **boa prática fundamental**: a camada de apresentação pode evoluir de forma independente da lógica da aplicação.



Resultado



Layout Responsivo

Grid adaptável para mobile, tablet e desktop com breakpoints bem definidos.



Interface Profissional

Visual limpo, organizado e alinhado com padrões modernos de design.



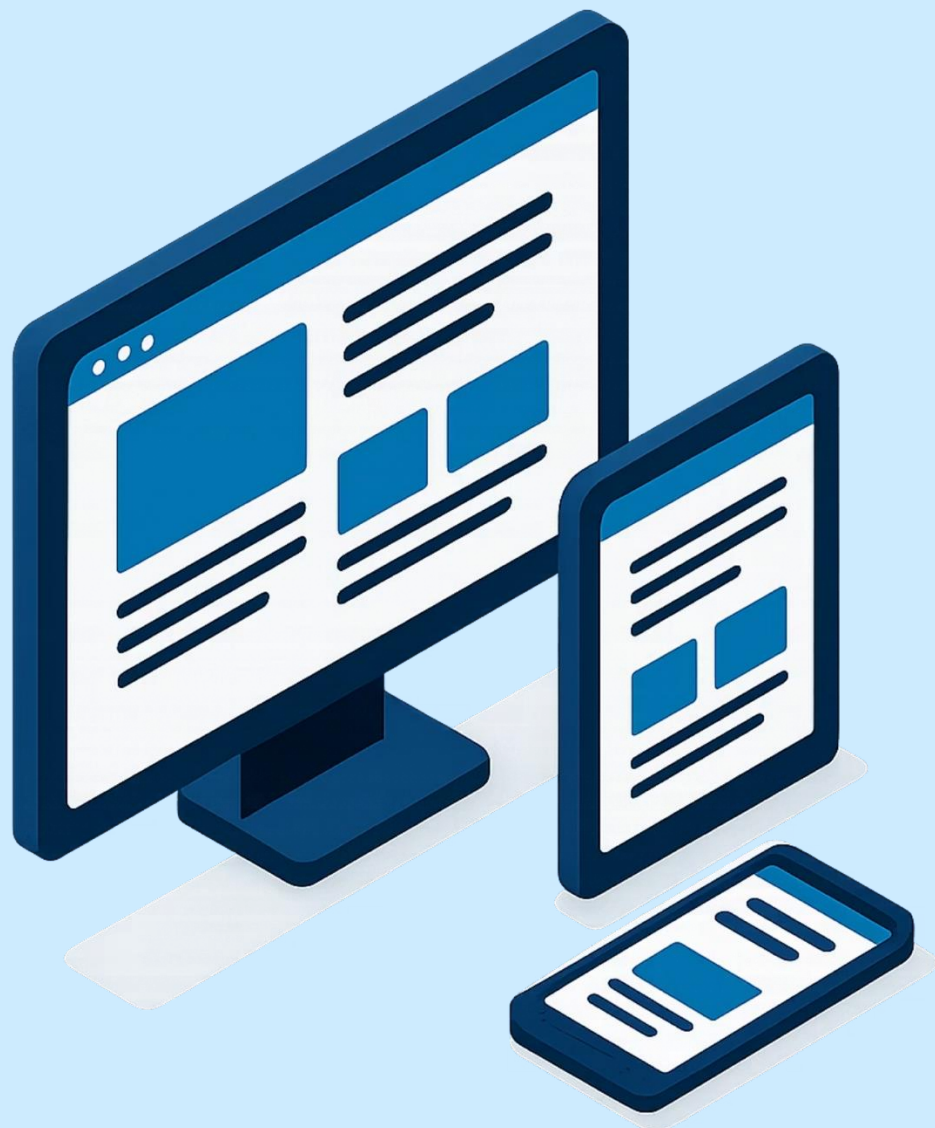
Melhor Organização Visual

Hierarquia clara, espaçamentos consistentes e tipografia padronizada.



Melhor UX

Experiência do usuário aprimorada com hierarquia de botões e layout adaptativo.



Conclusão da Aula

Nesta aula você aprendeu a evoluir uma aplicação Angular para um padrão responsivo e profissional:



Criar layouts responsivos

Estruturar o HTML com classes preparadas para grid CSS moderno.



Aplicar CSS moderno com grid

Usar `display: grid` e `grid-template-columns` para layouts flexíveis.



Utilizar breakpoints

Definir pontos de quebra com `@media` para adaptar o layout a cada dispositivo.



Melhorar UX sem alterar lógica

Separar responsabilidades: evoluir a apresentação sem tocar na lógica de negócio.



Todas as melhorias foram feitas apenas com HTML e CSS.



Parabéns! Sua aplicação Angular agora possui uma interface responsiva e profissional, pronta para ser usada em qualquer dispositivo.

Dúvidas

?

